

IDENTITY AND ACCESS MANAGEMENT

[Kaynak](#)

İÇERİK

1. IAAA Model: Güvenliğin 4 Temel Direği
2. IAAA Model: Identification (Kimlik Belirleme)
3. IAAA Model: Authentication (Kimlik Doğrulama)
4. IAAA Model: Authorisation & Access Control
5. IAAA Model: Accountability & Logging (Hesap Verilebilirlik)
6. Identity Management (IdM) vs. Identity and Access Management (IAM)
7. Attacks Against Authentication (Kimlik Doğrulama Saldırıları)
8. Access Control Models (Erişim Kontrol Modelleri)
9. Single Sign-On (SSO) - Tek Oturum Açma

IAAA Model: Güvenliğin 4 Temel Direği

Bilgi güvenliğinin temel amacı olan CIA Triad'ı (Confidentiality, Integrity, Availability) korumak için uyguladığımız sürecin adı **IAAA**'dır. Bu model, bir kullanıcının sisteme girişinden çıkışına kadar olan yaşam döngüsünü tanımlar.

IAAA şunların kısaltmasıdır:

1. Identification (Kimlik Belirleme)
2. Authentication (Kimlik Doğrulama)
3. Authorisation (Yetkilendirme)
4. Accountability (Hesap Verilebilirlik)

Bu dört aşama, yetkisiz erişimleri (unauthorised access) ve veri ihlallerini (data breaches) önlemek için sırasıyla uygulanır.

1. Identification (Kimlik Belirleme)

Soru: "Kimsin?"

Sürecin ilk adımıdır. Kullanıcının sisteme "Ben şu kişiyim" diyerek bir kimlik **iddia etmesidir (claiming an identity)**.

- **Nasıl Çalışır:** Kullanıcı benzersiz bir tanımlayıcı (Unique Identifier) sunar.
- **Örnekler:**
 - Kullanıcı Adı (Username)

- E-posta adresi (Birçok site kullanıcı adı yerine bunu tercih eder çünkü doğası gereği benzersizdir).
- TC Kimlik No veya Personel ID numarası.
- **Kritik Nokta:** Identifier, o ortamda **benzersiz (unique)** olmak zorundadır. İki kişiye aynı ID verilemez.
- **Not:** Bu aşamada henüz hiçbir doğrulama yoktur, sadece beyan vardır.

2. Authentication (Kimlik Doğrulama)

Soru: "Kanıtla."

Kullanıcının iddia ettiği kişinin **gerçekten o kişi olduğunun** doğrulanması sürecidir. Identification aşamasında yapılan iddianın ispatıdır.

- **Yöntemler:**
 - **Parola (Password):** En yaygın yöntem.
 - **MFA/2FA:** Parolaların zayıflığı nedeniyle, e-postaya veya telefona gelen kodlar gibi ek yöntemler popülerleşmiştir.
- **Mantık:** Eğer Identification "Ben Alice'im" demekse, Authentication "İşte bu da Alice olduğumun kanıtı (parolam)" demektir.

3. Authorisation (Yetkilendirme)

Soru: "Burada ne yapabilirsin?"

Kullanıcı kimliğini kanıtladıktan (Authentication) sonra, sistem içinde **hangi kaynaklara erişebileceğinin ve neleri yapabileceğinin** belirlenmesidir.

- **Nasıl Çalışır:** Kullanıcının rolüne, iş tanımına veya güvenlik seviyesine (clearance level) göre izinler (permissions) atanır.
- **Amaç:** Kullanıcıya sadece işini yapması için gereken **en az yetkiyi** (Least Privilege) vermek.
- **Örnek:**
 - Bir şirkette herkes "Giriş" yapabilir (Authentication).
 - Ancak sadece İK personeli maaş dosyalarını görebilir (Authorisation).
 - BT ekibi sunucuları yönetebilir ama finans verilerini göremez (Authorisation).

DİKKAT (Sınav Sorusu): Authentication ve Authorisation çok karıştırılır.

- **Authentication:** Kimlik kontrolü (Pasaport kontrolü).
- **Authorisation:** Erişim izni (Vizenin kapsadığı yerler).

4. Accountability (Hesap Verilebilirlik)

Soru: "Ne yaptın?"

Kullanıcı sisteme girdikten ve işlem yapmaya başladıktan sonra, yaptığı her hareketin **kayıt altına alınması** ve bu eylemlerden sorumlu tutulabilmesidir.

- **Mekanizma: Logging** (Loglama/Kayıt Tutma).
- **Nasıl Çalışır:** Tüm kullanıcı aktiviteleri merkezi bir lokasyonda loglanır.
- **Neden Önemli?**
 - Bir güvenlik ihlali (security incident) yaşandığında, olayın kaynağını bulmak (forensics).
 - Sorunun kimden kaynaklandığını tespit etmek.
 - İnkâr edilemezlik (Non-repudiation) sağlamak.

Özet Akış Senaryosu

Bir sisteme giriş yaparken beyinde şu zinciri kur:

1. **Identification:** Kullanıcı adı (admin) yazdım. -> *"Ben adminim."*
2. **Authentication:** Parolamı (12345) girdim. -> *"Al bu da kanıtım."*
3. **Authorisation:** Sistem bana Dashboard 'a erişim izni verdi ama Database 'i silmeme izin vermedi. -> *"Yetkin bu kadar."*
4. **Accountability:** Sistem log dosyasına [User: admin] accessed [Dashboard] at 10:00 PM yazdı. -> *"Ne yaptığımı kaydettim, sorumluluk sende."*

IAAA Model: Identification (Kimlik Belirleme)

IAAA modelinin ilk ve en temel basamağı olan **Identification**, bir kullanıcının (veya bir sürecin/sistemin) kimliğini **iddia etmesi (claiming a specific identity)** durumudur.

Bu aşamada henüz bir doğrulama veya kanıt sunma yoktur; sadece **"Ben şuyum"** beyanı vardır.

1. Mantiğı Anlamak: Sosyal Ortam vs. Güvenlik

Kavramı oturtmak için iki farklı senaryoyu inceleyelim:

A. Parti Senaryosu (Sadece Identification)

Bir partiye gittin ve biri adını sordu.

- "Adım Thomas Anderson" dedin.
- Hatta şaka yapıp "Ben Neo" dedin.
- **Sonuç:** Karşıdaki kişi kimlik kartını sormaz. Sadece sosyalleşıyorsunuz, yüksek güvenliqli bir alana girmiyorsunuz. Beyanın (Identification) yeterli kabul edilir.

B. Spor Salonu Senaryosu (Identification + Authentication Gereksinimi)

Spor salonuna gittin ve resepsiyona "Adım Thomas Anderson, üyeyim" dedin.

- Resepsiyonist sadece bu beyanla seni içeri almaz. Çünkü herkes başkasının ismini söyleyip bedava girebilir.
- Senden **Kimlik Kartını** veya **Parmak İzini** ister.
- İşte bu ikinci aşama (kanıt isteme) bir sonraki konu olan **Authentication**'dir.

Kritik Ayrım: Identification, "Ben aboneyim" demektir. Eğer sistem sadece Identification ile çalışsaydı (Authentication olmadan), işletmeler batır, bankalardan sahte kimlikle kredi çekilirdi ve sadece e-posta adresini bilen herkes maillerini okuyabilirdi.

2. Tanımlayıcı Türleri (Identifiers)

Siber dünyada Identification için kullanılan verinin o ortamda **Eşsiz (Unique)** olması gerekir.

A. Username (Kullanıcı Adı)

Kurumun veya platformun politikasına göre değişir. "Thomas Anderson" için olası varyasyonlar:

Plaintext

```
tanderson  
thomasa  
thomas01  
ta001  
neo
```

B. Sayısal Tanımlayıcılar (Numeric IDs)

İsim benzerliklerini önlemek için kesin çözüm sunan numaralardır:

- TC Kimlik Numarası (National ID)
- Öğrenci Numarası
- Pasaport Numarası
- Cep Telefonu Numarası

C. E-Posta Adresi (En Yaygın Yöntem)

Günümüzde birçok web sitesi kullanıcı adı yerine e-posta ister.

- **Neden?** Çünkü e-posta sisteminin doğası gereği, bir e-posta adresi dünya çapında **benzersizdir (guaranteed to be unique)**.
- Kullanıcıyı "Bu kullanıcı adı alınmış, başka dene" derdinden kurtarır.

3. Güvenlik Riski: Authentication Olmadan Identification

Eğer Identification tek başına kullanılsaydı (yani Authentication adımı atlanıp sadece beyana güvenileseydi):

- **E-posta:** Senin e-posta adresini (public bilgi) bilen herkes gelen kutuna erişirdi.
- **Bankacılık:** Başkasının adını söyleyen herkes onun adına kredi çekerdi.
- **Sistemler:** Sadece bilgisayarlar değil; otel rezervasyon sistemleri, uçuş sistemleri vb. çökerdi.

Özet: Identification, "Ben kimim?" sorusuna verilen cevaptır. Ancak bu cevabın doğruluğu bir sonraki adım olan **Authentication** ile test edilmeden hiçbir güvenlik değeri taşımaz.

IAAA Model: Authentication (Kimlik Doğrulama)

Önceki adımda "Ben şuyum" demiştik (Identification). Şimdi ise **Authentication** ile "İşte kanıtım" diyoruz. Bu aşama, iddia edilen kimliğin doğrulanması sürecidir.

Spor salonu örneğine dönersek:

- **Identification:** "Ben üyeyim" demek.
- **Authentication:** Üyelik kartını (fotoğraflı ve çipli) gösterip kapıyı açmak. Kartın sahte olmaması şartıyla kimlik doğrulanmış olur.

Siber güvenlikte Authentication genellikle şu 3 ana faktörden (Factors of Authentication) biri veya birkaçıyla yapılır:

1. Authentication Faktörleri (Kimlik Doğrulama Türleri)

Bir kullanıcının kimliğini kanıtlaması için kullanılan yöntemler 3 ana başlıkta toplanır (+2 yan başlık).

A. Something You Know (Bildiğin Bir Şey)

En yaygın ve geleneksel yöntemdir. Beyninde tuttuğun, ezberlediğin bir bilgidir.

- **Passwords (Parolalar):** Karmaşık karakter dizileri. Ör: 4\$NoPawKkdFiCdnm
 - **Passphrases (Parola İfadeleri):** Kelimelerden oluşan uzun cümleler. Ör: Judge Battle Advise Pain 9 . (Genelde daha güvenlidir).
 - **PIN (Personal Identification Number):** Sayısal şifreler. Ör: 25063 .
 - **Pattern (Desen Kilidi):** Telefonlarda parmakla çizilen şekil.
- **Not:** Desen kilidi teknik olarak PIN'den farksızdır. Çünkü o şekli de "ezberlersin" (something memorised).

Örnek (TryHackMe Login):

- **Username/Email:** Identification (Kimlik Belirleme - Herkes bilebilir).

- **Password:** Authentication (Sadece sen biliyorsun, hesabın sahibi olduğunu kanıtlar).

B. Something You Have (Sahip Olduğun Bir Şey)

Fiziksel olarak elinde tuttuğun bir nesnedir.

- **Telefon / SIM Kart:** WhatsApp veya Telegram kurulumunda gelen SMS kodu.
 - Mantık: "Eğer bu numaraya gelen SMS'i okuyabiliyorsan, o SIM kart fiziksel olarak elindedir."
- **Hardware Security Keys (Donanım Güvenlik Anahtarları):** Anahtarlığında taşıdığın, USB veya NFC ile çalışan özel cihazlar.
 - **Markalar:** Yubico (YubiKey), Titan Security Key (Google), Nitrokey, Thetis.
 - Çalışma şekli: Cihazı bilgisayara takarsın veya telefona dokundurursun.

C. Something You Are (Sen Olan Bir Şey - Biyometrik)

Kişinin fiziksel özellikleridir (Biometrics). Taklit edilmesi en zor olan faktördür.

- **Fingerprint (Parmak İzi):** En yaygın olanı.
- **Facial Recognition (Yüz Tanıma):** Modern telefonlardaki FaceID vb.
- **Retina Scanners (Retina Taraması):** Gözdeki damar yapısı.
- **Voice Recognition (Ses Tanıma).**

Pratik Not: Telefonlar genelde parmak izi okumasa bile yedek olarak PIN ister. Yani "Something You Are" çalışmazsa "Something You Know" devreye girer.

Diğer Faktörler (Daha az kullanılır)

- **Somewhere You Are (Olduğun Yer):** Fiziksel veya mantıksal konum (Örn: Sadece şirket binasındaki IP bloğundan girişe izin vermek).
- **Something You Do (Yaptığın Şey - Davranış):** Yazı yazma hızı, fareyi hareket ettirme şekli vb. (Behavioural Biometrics).

2. Multi-Factor Authentication (MFA & 2FA)

Tek bir faktör (örneğin sadece parola) güvenli değildir çünkü çalınabilir. Güvenliği artırmak için birden fazla faktörü birleştiririz.

Tanım

- **2FA (Two-Factor Authentication):** Yukarıdaki faktörlerden **tam olarak ikisinin** kullanılmasıdır.
- **MFA (Multi-Factor Authentication):** İki **veya daha fazla** faktörün kullanılmasıdır. (Her 2FA bir MFA'dır).

Klasik Örnek: ATM (Bankamatik)

ATM, dünyadaki en eski ve sağlam 2FA örneklerinden biridir:

1. **Kartı takarsın:** Something You Have (Sahip olduğun).
 2. **Şifreyi girersin:** Something You Know (Bildiğin).
- **Senaryo:** Hırsız kartını çalsa bile (Have), şifreni (Know) bilmediği için para çekemez. Sadece şifreni bilse, kart elinde olmadığı için yine çekemez.

Ev Kasası Örneği

Anahtarlı ve şifreli kasalar:

- Anahtar tak (Have) + PIN gir (Know).

Kritik Çıkarım: MFA, zayıf parola riskini minimize eder. Saldırgan parolanı (Know) ele geçirse bile, telefonundaki authenticator app'e (Have) veya parmak izine (Are) sahip olmadığı için hesaba giremez.

Soru Çözüm Anahtarı (Task Referansı)

Sorularda şu numaraları kullanacağız:

1. **Something you know** (Parola, PIN, Annenin kızlık soyadı vb.)
2. **Something you have** (Telefon, Akıllı Kart, YubiKey)
3. **Something you are** (Parmak izi, Yüz)
4. **2FA** (İki faktörün birleşimi)

IAAA Model: Authorisation & Access Control

Authentication adımını geçtik, sistem kim olduğumuzu (Identity) doğruladı. Şimdi en kritik soruya geliyoruz: **"Bu kimlik ile bu sistemde neler yapabilirim?"**

Bu bölümde birbirine sıkı sıkıya bağlı ama farklı iki kavramı netleştiriyoruz: **Authorisation** (Yetkilendirme) ve **Access Control** (Erişim Kontrolü).

1. Authorisation (Yetkilendirme)

Tanım: **Kural Kitabı (The Policy)**

Kullanıcının kimliği doğrulandıktan sonra, hangi kaynaklara erişebileceğinin ve hangi işlemleri yapabileceğinin **belirlenmesidir**. Bu bir **karardır**, bir politika.

- **Spor Salonu Örneği:**
 - Ücretini ödeyen bir üye, çalışma saatleri içinde koşu bandını kullanabilir. (Yetkisi var).
 - Ancak aynı üye, koşu bandını sırtlayıp "Bunu bir ay evde kullanacağım" diyerek eve götüremez. (Yetkisi yok).

- Spor salonu kütüphane değildir; neyin yapıp yapılamayacağı kurallarla (Authorisation) bellidir.
- **Otel Örneği:**
 - Linda otele geldi ve 1 hafta oda tuttu.
 - Linda'nın yetkisi: Kendi odasına girmek ve lobi gibi ortak alanları kullanmak.
 - Linda'nın yetkisizliği: Diğer misafirlerin odasına girmek.

2. Access Control (Erişim Kontrolü)

Tanım: Uygulayıcı (The Enforcement)

Authorisation aşamasında belirlenen kuralların (politikaların) **teknik veya fiziksel mekanizmalarla uygulanmasıdır**. Yetkilendirme "karar verir", Erişim Kontrolü "uygular".

- **Spor Salonu Örneği:**
 - Eğer üye koşu bandını çalmaya kalkarsa, kapıdaki görevlinin onu durdurması veya turnikeden geçirmemesi **Access Control** mekanizmasıdır.
- **Otel Örneği:**
 - Linda'nın "kendi odasına girme yetkisi" (Authorisation) vardır.
 - Bu yetkiyi uygulayan şey, resepsiyonun verdiği **Elektronik Kart (Key Card)** ve kapıdaki **Kilit**dir.
 - Kapı kilidi, Linda'nın kartını okur ve açılır; başkasının odasına denerse açılmaz. İşte bu mekanizma Access Control'dür.

3. Teknik Dünyada Karşılığı

Bir şirket senaryosu düşünelim (Edward, Satış Ekibinde):

- **Senaryo:** Edward'ın işini yapabilmesi için satış dosyalarına (Sales Files) erişmesi gerekir. Ancak İnsan Kaynakları (HR) veya Muhasebe (Accounting) dosyalarını görmesine gerek yoktur.
- **Authorisation (Politika):** "Satış ekibi üyeleri sadece /sales klasörüne erişebilir, /hr klasörüne erişemez." kararıdır.
- **Access Control (Mekanizma):** Sistem yöneticisinin sunucuda dosya izinlerini (File Permissions) chmod veya NTFS izinleriyle ayarlamasıdır. Edward /hr klasörüne tıkladığında "Access Denied" hatası alıyorsa, Access Control mekanizması çalışıyor demektir.

E-Posta Sunucusu Örneği

- Gmail'e girdiğinde kendi gelen kutunu okuyabilirsin, yeni mail atabilirsin.
- Ama iş arkadaşının gelen kutusunu göremezsin.
- Mail sunucusu (Server), senin sadece kendi kutuna (mailbox) erişmene izin verecek, diğerlerini reddedecek şekilde tasarlanmıştır.

4. Kritik Ayırım (Sınav Notu)

Bu iki kavramı birbirinden ayırmak için şu basit kuralı kullan:

Kavram	Soru	Metafor
Authorisation	"Neye izni var?"	Kural Kitabı / Yasa
Access Control	"Bunu nasıl engellerim/sağlarım?"	Kilit / Polis / Turnike

- **Authorisation:** Bir strateji ve karardır. (Kimin nereye gireceğini belirlemek).
- **Access Control:** Bu kararı uygulayan araçtır. (Kilitler, İzinler, Güvenlik Duvarları).

IAAA Model: Accountability & Logging (Hesap Verilebilirlik)

IAAA modelinin son halkasıdır. Kullanıcı kimliğini doğruladı (Authentication), yetkisini aldı (Authorisation) ve sisteme girdi. Peki, içeride ne yaptı? Bir hata yaparsa veya kötü niyetli bir işlem gerçekleştirirse bunun sorumlusu kim?

Accountability (Hesap Verilebilirlik): Kullanıcıların sistem üzerindeki eylemlerinden sorumlu tutulabilmesidir.

- **Ön Koşul:** Bunu sağlayabilmek için **Auditing** (Denetim) yeteneğine ve düzgün çalışan bir **Logging** (Kayıt Tutma) mekanizmasına ihtiyacımız vardır.

1. Analoji ile Anlayalım

Spor Salonu Örneği

Salona girdin, kimse kimliğini sormadı (zaten tanıyorlar). İçeridesin.

- Eğer sinirlenip aynayı kırarsan, salon yönetimi senin üyeliğini iptal eder ve zararı sana ödetir.
- Neden? Çünkü o an orada olan ve o eylemi yapanın "SEN" olduğu bellidir. Herkes eylemlerinden sorumludur.

Banka Gişesi Örneği (Teknik)

Bir banka çalışanı müşterinin hesabına girip para transferi yapabilir (Yetkisi var).

- **Risk:** Ya bu çalışan kötü niyetliyse ve müşterinin parasını çalarsa?
- **Çözüm:** Banka sistemi, yapılan her işlemi (Transaction) saniyesi saniyesine kaydeder.
 - *Kim yaptı?* Gişe Memuru Ahmet.
 - *Ne yaptı?* 10.000 TL transfer.
 - *Ne zaman?* 14:02.

- Bu kayıtlar (Loglar) sayesinde sistem güvenilirdir. Kayıt yoksa güven de yoktur.

2. Logging (Kayıt Tutma)

Logging, sistemde gerçekleşen olayların (Events) kaydedilmesi sürecidir. Sadece kullanıcı hareketleri değil, sistem hataları ve uyarılar da kaydedilir.

Neden Log Tutarız?

1. **Regulatory Compliance (Yasal Uyumluluk):** Yasalar (KVKK, GDPR, PCI-DSS) şirketlerin log tutmasını zorunlu kılar.
2. **Incident Response (Olay Müdahalesi):** Bir saldırı anında "Şu an ne oluyor?" sorusunun cevabı loglardadır.
3. **Forensic Investigations (Adli Bilişim):** Saldırı bittikten sonra "Saldırgan nereden girdi, neyi çaldı?" sorularını cevaplamak için geçmişe dönük loglar incelenir.

Örnek Senaryo:

Hassas bir veriye yetkisiz erişim denemesi olduğunda, log sistemi bunu kaydeder ve güvenlik ekibine **alarm (alert)** üretir.

3. Log Forwarding (Log Yönlendirme) & Log Güvenliği

Bir saldırgan sisteme sızdığı anda yapacağı **İLK ŞEY** nedir?

İzlerini silmek için log dosyalarını temizlemek.

Eğer loglar sadece saldırıya uğrayan sunucuda (Local) tutuluyorsa, saldırgan onları silebilir (`rm -rf /var/log`). Buna karşı savunma mekanizmamız **Log Forwarding**'dir.

- **Log Forwarding:** Logların oluştuğu makineden anlık olarak alınıp, merkezi ve güvenli başka bir sunucuya (Centralised Location) veya Buluta (Cloud) gönderilmesidir.
- **Tamper-Proof (Korcalanamaz):** Saldırgan sunucuyu ele geçirse bile, loglar çoktan başka bir sunucuya gönderildiği için geçmişi silemez.
- **Best Practice:** Loglama sunucusu (Logging Server) sadece log alır, başka hiçbir iş yapmaz ve çok sıkı korunur.

4. SIEM (Security Information and Event Management)

Yüzlerce sunucudan, güvenlik duvarından ve uygulamadan milyonlarca satır log akıyor. Bunları insan gözüyle takip etmek imkansızdır. İşte burada **SIEM** devreye girer.

- **Tanım:** Farklı kaynaklardan gelen log verilerini tek bir merkezde toplayan (aggregate), analiz eden ve güvenlik tehditlerini tespit eden teknolojidir.
- **Ne Yapar?**
 - **Anomali Tespiti:** "Bu kullanıcı normalde İstanbul'dan giriyor, aynı anda neden New York'tan giriş denemesi var?"

- **Korelasyon (Correlation):** "Firewall'da port taraması oldu + 1 dakika sonra Sunucuda başarısız girişler arttı + 2 dakika sonra Admin girişi yapıldı" → **KIRMIZI ALARM!**
- **Faydaları:**
 - Olaylara hızlı müdahale.
 - Denetimler için hazır raporlama (Compliance Reporting).
 - Detaylı adli analiz imkanı.

Özet Zincir

1. **Kullanıcı:** Eylemi yapar.
2. **Sistem:** Eylemi Loglar (Logging).
3. **Log Forwarding:** Logu güvenli kasaya gönderir.
4. **SIEM:** Logu okur, analiz eder ve gerekirse alarm çalar.
5. **Accountability:** Kullanıcı eyleminden sorumlu tutulur.

Identity Management (IdM) vs. Identity and Access Management (IAM)

Bu notlar, genellikle birbirine karıştırılan ancak siber güvenlik mimarisinde kritik öneme sahip iki kavramı netleştirmek için çıkarıldı: **IdM** ve **IAM**.

Temel amaç: "Doğru kişilerin doğru kaynaklara doğru zamanda erişmesini sağlamak, yanlış kişileri ise kapıda bırakmak."

1. Temel Kavramlar ve Farklılıklar

Literatürde bu iki terim bazen birbirinin yerine kullanılsa da, teknik olarak aralarında bir nüans vardır. Bu ayrımı bilmek sistem tasarımı açısından önemlidir.

- **IdM (Identity Management):** Odak noktası "**Kimlik**"tir.
 - Kullanıcıların, cihazların ve grupların özelliklerini (attributes) ve izinlerini yönetir.
 - Kullanıcı veritabanını, kimlik bilgilerini tutan ve yöneten yapıdır.
 - *Soru:* "Bu kullanıcının kimlik bilgileri güncel mi? Hangi departmanda?"
- **IAM (Identity and Access Management):** Odak noktası "**Erişim ve Politika**"dır.
 - IdM'den daha kapsamlıdır.
 - IdM'deki kimlik bilgilerini alır, şirket politikalarına göre değerlendirir ve erişimi **onaylar** veya **reddeder**.
 - Yönetişim (Governance) ve Uyumluluk (Compliance) buranın konusudur.

Özet Ayrım: IdM, kimlik verisinin "yönetimiyle" ilgilenirken; IAM, bu veriyi kullanarak erişimin "güvenliğini sağlamakla" ve denetlemekle ilgilenir.

2. Identity Management (IdM) Detayları

IdM, siber güvenliğin **kimlik yönetimi** bileşenidir. Organizasyon içindeki her kullanıcıya veya cihaza bir **Digital Identity (Dijital Kimlik)** atanmasını sağlar.

Nasıl Çalışır?

- **Merkezi Veritabanı (Centralised Database):** Tüm kullanıcı kimlikleri ve erişim hakları tek bir merkezde tutulur.
- **Single Point of Reference:** Kullanıcı yönetimi için tek bir referans noktası sağlar. 10 farklı sisteme ayrı ayrı kullanıcı açmak yerine IdM üzerinden yönetilir.

Temel Fonksiyonları

1. **User Provisioning (Kullanıcı Sağlama):** Kullanıcı hesaplarının oluşturulması, güncellenmesi ve yaşam döngüsünün yönetilmesi sürecidir. (Örn: İşe yeni giren biri için hesap açılması).
2. **Authentication & Authorisation:** Kullanıcının kimliğinin doğrulanması ve yetkilendirilmesi için altyapı sağlar.

Avantajları

- Kullanıcı erişim süreçlerini otomatikleştirir (Streamline).
- Kimlik yönetimi maliyetlerini düşürür.
- Kullanıcı deneyimini (UX) iyileştirir (Sürekli farklı şifrelerle uğraşmazlar).

3. Identity and Access Management (IAM) Detayları

IAM, IdM'i de içine alan çok daha geniş bir şemsiyedir. Dijital kimliklerin güvenliğini, yaşam döngüsünü ve uyumluluğunu yöneten teknolojiler bütünüdür.

Temel Teknolojiler ve Yöntemler

IAM sistemleri güvenliği sağlamak için şu teknolojileri entegre eder:

- **RBAC (Role-Based Access Control):** Erişimin, kişinin kimliğine göre değil, şirketteki **rolüne** (Örn: Muhasebe, İK, IT) göre verilmesidir.
- **MFA (Multi-Factor Authentication):** Erişim için birden fazla doğrulama faktörü istenir.
- **SSO (Single Sign-On):** Kullanıcının tek bir kez oturum açarak, yetkisi olan tüm uygulamalara şifre girmeden erişebilmesidir.

Lifecycle Management (Yaşam Döngüsü Yönetimi)

IAM, bir kullanıcının şirkete girişinden çıkışına kadar olan süreci yönetir:

1. **Onboarding:** İşe giriş, hesapların ve yetkilerin otomatik tanımlanması.
2. **Access Governance:** Yetkilerin periyodik kontrolü.

3. **Offboarding / Access Revocation:** İşten çıkış anında tüm yetkilerin **anında** geri alınması. (Güvenlik ihlallerini önlemek için en kritik adım).

Compliance (Uyumluluk) ve Denetim

IAM sistemleri, yasal düzenlemelere (**GDPR, HIPAA, PCI DSS**) uyum sağlamak için zorunludur.

- **Auditing (Denetim):** "Kim, ne zaman, hangi kaynağa erişti?" sorusunun cevabını verir.
- Güvenlik ihlallerini (Security Breaches) önlemek ve olay sonrası analiz (Forensics) için log tutar.

Karşılaştırma Özeti

Özellik	IdM (Identity Management)	IAM (Identity & Access Management)
Odak	Kimlik verisi ve yönetimi	Erişim kontrolü, politika ve güvenlik
Kapsam	Daha dar (Operasyonel)	Daha geniş (Stratejik ve Güvenlik)
Ana Görev	Hesap açma, kimlik tutma	Kimlik yönetimi + Erişim denetimi + Uyumluluk
İlişki	IAM'in bir alt bileşenidir	IdM'i kapsayan üst kümedir

Attacks Against Authentication (Kimlik Doğrulama Saldırıları)

Bu bölümde, "Neden kendi kafamıza göre bir kimlik doğrulama protokolü yazmamalıyız?" sorusunun cevabını veriyoruz. Test edilmemiş, "naive" (saf/basit) protokollerin nasıl kolayca atlatılabileceğini (Bypass) ve standart protokollerin (Kerberos, TLS vb.) neden hayati olduğunu göreceğiz.

Temel kural: **"Don't roll your own crypto/protocol."** (Kendi kriptonu/protokolünü yazma, standartları kullan).

1. Fiziksel Dünyada Kimlik Doğrulama (Analogue World)

Senaryo üzerinden gidelim: Bir at binme kulübündeyiz. Haftalık yemekler için bir restoranı kapatıyoruz ve kapıda bir güvenlik görevlisi var. Görevli herkesi tanımıyor, bu yüzden bir "parola" sistemi kuruyoruz.

Yöntem 1: Sabit Parola (Secret Phrase)

- **Mekanizma:** Kapıdaki soruyor: "How many?", Üye cevaplıyor: "Seven horses".
- **Saldırı (Eavesdropping / Kulak Misafiri Olma):**
 - Saldırgan kapıya yakın bir yerde bekler.

- Gerçek bir üyenin "Seven horses" dediğini **duyar**.
- Artık saldırgan da parolayı biliyordur ve elini kolunu sallayarak içeri girer.
- *Sonuç*: Başarısız.

Yöntem 2: Çoklu Soru-Cevap

- **Mekanizma**: Tek soru yerine 10 farklı soru ve 10 farklı cevap belirleriz.
- **Saldırı**: Saldırgan yeterince uzun süre kapıda beklerse (Sniffing), 10 sorunun da cevabını zamanla öğrenir.
- **Kritik Çıkarım**: Kriptografi kullanmadan sadece "gizli kelimelerle" güvenli bir sistem kurmak neredeyse imkansızdır. Fiziksel dünyada şüpheli tipleri kovabilirsiniz ama dijital dünyada saldırgan görünmezdir.

2. Dijital Dünyada Saldırılar ve Replay Attack

Ağ üzerindeki durum çok daha kritiktir.

A. Cleartext (Açık Metin) İletişim

Kullanıcı adını ve parolasını şifrelemeden gönderirsen, ağı dinleyen (Wireshark vb. kullanan) herkes bu bilgileri olduğu gibi okur. Bu senaryo tartışmaya bile kapalıdır; **güvensizdir**.

B. Sabit Anahtarlı Şifreleme (The Naive Encryption)

Diyelim ki sunucu ile aramızda sabit bir gizli anahtar (Secret Key) belirledik. Parolamızı bu anahtarla şifreleyip gönderiyoruz.

- **Senaryo**:
 - Parola: `P4ssw0rd`
 - Şifreli Hali (Ciphertext): `Xy9#m2!z`
 - Kullanıcı giriş yaparken her seferinde sunucuya `Xy9#m2!z` gönderiyor.
- **Saldırı: Replay Attack (Tekrar Saldırısı)**
 - Saldırgan ağı dinler ve `Xy9#m2!z` verisini yakalar (Capture).
 - Saldırganın bu şifreyi çözmesine veya gerçek parolanın `P4ssw0rd` olduğunu bilmesine **gerek yoktur**.
 - Saldırgan, yakaladığı `Xy9#m2!z` paketini sunucuya tekrar gönderir (Replay).
 - Sunucu şifreyi çözer, bakar ki parola doğru, saldırganı içeri alır.

Mantığı Anla: Bu, bir konserin giriş biletini kopyalamak gibidir. Biletin üzerindeki barkodun (şifreli verinin) ne anlama geldiğini bilmene gerek yok; fotokopisini çekip (Capture) kapıdaki görevliye uzatırsan (Replay) içeri girersin.

3. Çözüm: Benzersiz Cevaplar (Making the Response Unique)

Replay Attack'ı engellemenin yolu, gönderilen şifreli verinin **her seferinde değişmesini** sağlamaktır. Böylece saldırgan eski paketi tekrar gönderdiğinde sunucu "Bu kullanılmış/eski bir paket" diyerek reddeder.

Yöntem: Timestamp (Zaman Damgası) veya Nonce

Şifreleme işlemine "zamanı" dahil ederiz.

- **Yeni Mekanizma:**
 - Kullanıcı sadece parolayı şifrelemez. `Encrypt(Password + Current Time)` yapar.
 - Örnek:
 - Saat 10:00:01 → Şifreli Veri: A1b2...
 - Saat 10:00:02 → Şifreli Veri: C9d8...
 - Sunucu paketi aldığı anda şifreyi çözer ve içindeki saate bakar.
- **Güvenlik Analizi:**
 - Saldırgan A1b2... paketini yakaladı (Saat 10:00:01).
 - Saldırgan 1 dakika sonra bu paketi tekrar gönderdi (Replay).
 - Sunucu paketi açtı, içindeki saate baktı: "Saat 10:00:01".
 - Sunucu kendi saatine baktı: "Şu an 10:01:00".
 - **Sonuç:** "Bu paketin süresi geçmiş, kabul etmiyorum." (Authentication Failed).

Not: Bu yöntemin çalışması için Sunucu (Server) ve İstemci (Client) saatlerinin **senkronize** olması (NTP - Network Time Protocol) gerekir. Paketlerin geçerlilik süresi genellikle milisaniyelerle sınırlıdır.

Özet Tablo

Yöntem	Salırgan Ne Görür?	Salı Türü	Sonuç
Cleartext	username:admin , pass:1234	Sniffing	✗ Parola Çalındı
Static Encryption	Enc(1234) -> Xy9#...	Replay Attack	✗ Giriş Yapıldı (Parola bilinmese bile)
Time-based Encryption	Enc(1234 + Time) -> Değişken	Replay Attack	✓ Başarısız (Süresi geçmiş paket)

Access Control Models (Erişim Kontrol Modelleri)

Sistemlerin kaynaklara (dosya, yazıcı, veritabanı vb.) kimin erişeceğine karar verirken kullandığı 3 temel model vardır. Bu modeller siber güvenlik mimarisinin bel kemiğidir.

Task'teki soruları cevaplamak için şu numaraları kullanacağız:

1. **DAC (Discretionary Access Control)**
2. **RBAC (Role-Based Access Control)**
3. **MAC (Mandatory Access Control)**

1. DAC (Discretionary Access Control)

Türkçesi: İsteğe Bağlı Erişim Kontrolü

En yaygın ve en esnek modeldir. Windows ve Linux dosya sistemlerinin varsayılan (default) davranışdır.

Çalışma Mantığı

- **Patron sensin (Owner is King):** Bir dosyanın veya kaynağın sahibi (Owner), o kaynağa kimin erişebileceğine **kendisi karar verir**.
- **Örnek:** Google Drive veya Dropbox'ta bir klasör açtın. Bu klasörü "Ahmet" ile paylaştın, "Ayşe"ye ise yasakladın. İzinleri tek tek sen (Data Owner) dağıttın.

Avantaj & Dezavantaj

- **(+) Esneklik:** Küçük gruplar ve kişisel kullanım için mükemmeldir. Hızlıca izin verip alabilirsin.
- **(-) Ölçeklenebilirlik Sorunu:** Şirkette 1000 çalışan olduğunu düşün. Her dosya için tek tek 1000 kişiye izin atamak imkansızdır. Ayrıca bir kişinin görevi değişirse (örn. Satıştan Muhasebeye geçerse), eski dosyalarındaki izinleri tek tek kaldırmak tam bir kâbustur.

2. RBAC (Role-Based Access Control)

Türkçesi: Rol Tabanlı Erişim Kontrolü

Kurumsal dünyada (Enterprises) standart olan modeldir. İzinler kişilere değil, **ROLLER** (gruplara) verilir.

Çalışma Mantığı

- **Kişi Yok, Rol Var:** "Ahmet"e izin vermezsin. "Muhasebeciler" grubuna izin verirsin. Ahmet'i o gruba eklersin.
- **Örnek:**
 - **Muhasebeci Rolü:** Muhasebe defterlerini görebilir, Ar-Ge (R&D) laboratuvarına giremez.
 - **Mühendis Rolü:** Ar-Ge laboratuvarına girer, muhasebe defterini göremez.
- **Senaryo:** Ahmet muhasebeden istifa edip mühendis olursa, tek yapman gereken onu "Muhasebe" grubundan çıkarıp "Mühendis" grubuna eklemektir. Tüm dosya izinleri otomatik güncellenir.

Avantaj

- **Yönetilebilirlik:** Büyük organizasyonlarda giriş-çıkış ve yetki değişikliği süreçlerini (User Provisioning) inanılmaz kolaylaştırır. Bakım maliyeti düşüktür.

3. MAC (Mandatory Access Control)

Türkçesi: Zorunlu Erişim Kontrolü

Güvenliğin en sıkı olduğu, "Paranoyak Modu" diyebileceğimiz modeldir. Askeri sistemler ve yüksek güvenlikli sunucular kullanır.

Çalışma Mantığı

- **Sistem Patron:** Kullanıcının veya dosya sahibinin **hiçbir söz hakkı yoktur**.
- **Etiketleme (Labeling):** Her nesneye (dosya) ve her özneye (kullanıcı) bir güvenlik etiketi (örn: "Çok Gizli", "Gizli", "Halka Açık") atanır.
- **Kural:** Kullanıcı istese bile, "Çok Gizli" etiketli bir dosyayı "Halka Açık" bir kullanıcıyla paylaşamaz. İzinleri değiştiremez (chmod çalışmaz). İşletim sistemi (Kernel) buna engel olur.
- **Kısıtlama:** Kullanıcılar yeni yazılım kuramaz, sistem ayarlarını değiştiremez. Sadece işlerini yapmak için gereken minimum haklara sahiptirler.

Linux Dünyasındaki Karşılıkları

Linux'ta MAC mekanizmasını uygulayan iki popüler modül vardır (Bunlar sınav sorusu potansiyelidir):

1. **AppArmor:** Debian ve Ubuntu tabanlı dağıtımlarda gelir.
2. **SELinux (Security Enhanced Linux):** Red Hat ve Fedora tabanlı dağıtımlarda standarttır. Daha karmaşık ve güçlüdür.

Özet Karşılaştırma Tablosu

Model	Karar Verici	Esneklik	Güvenlik	Kullanım Alanı
DAC	Veri Sahibi (User)	Yüksek	Düşük	Kişisel PC, Küçük Ofis
RBAC	Sistem Yöneticisi (Gruplar üzerinden)	Orta	Orta-Yüksek	Şirketler, Kurumsal
MAC	İşletim Sistemi / Politika	Yok	Çok Yüksek	Askeri, Kritik Sunucular

Single Sign-On (SSO) - Tek Oturum Açma

Kullanıcıların günlük işlerini yaparken e-posta, dosya sunucusu, yazıcı, İK portalı gibi onlarca farklı servise girmesi gerekir. Her biri için ayrı kullanıcı adı ve parola tutmak hem güvenlik

riskidir (çünkü kullanıcılar aynı şifreyi her yerde kullanmaya başlar) hem de yönetilemez bir durumdur.

SSO (Single Sign-On), bu kaosu bitiren teknolojidir.

1. Sorun Nedir? (Password Fatigue)

Geleneksel yöntemde senaryo şöyledir:

- Bilgisayarı açmak için 1. Şifre.
- Mail okumak için 2. Şifre.
- Dosya sunucusu için 3. Şifre.
- ...ve liste uzar gider.

Sonuç: Kullanıcılar bu kadar çok "güçlü" parolayı akılda tutamaz. Ya parolaları bir kağıda yazarlar (güvensiz) ya da her yerde 123456 kullanırlar (daha güvensiz).

2. Çözüm: SSO (Tek Bir Anahtar)

Mantık: Kullanıcı sisteme **sadece bir kez** (single set of credentials) kimliğini kanıtlar. Bu doğrulama (Authentication) başarılı olduktan sonra, yetkisi olan diğer tüm sistemlere (e-posta, dosya vb.) tekrar şifre girmeden otomatik erişir.

- *Analoji:* Otele girerken resepsiyonda bir kez kimlik gösterirsin, sana bir bileklik takarlar. O bileklikle havuza, restorana, odaya tekrar kimlik göstermeden girersin.

3. Avantajları (Neden Kullanıyoruz?)

SSO sadece kullanıcıya kolaylık değil, güvenlik mimarisine de büyük katkı sağlar:

1. Tek ve Güçlü Parola (One Strong Password):

- Kullanıcıdan 10 farklı karmaşık şifre ezberlemesini bekleyemezsin. Ama **tek bir** çok güçlü şifreyi (örn. 20 karakterli passphrase) ezberlemesini bekleyebilirsin. Odak noktası dağılmaz.

2. Kolay MFA Yönetimi (Easier MFA):

- Her uygulama için ayrı ayrı 2FA (SMS/Authenticator) kurmak bir kâbustur.
- SSO ile MFA'yı sadece giriş kapısına (Gateway) kurarsın. Kullanıcı girişte MFA'yı geçer, içerideki 50 uygulama otomatik güvenli hale gelir.

3. Daha Basit Destek (Simpler Support):

- IT ekiplerine gelen çağrıların büyük kısmı "Şifremi unuttum"dur. Hesap sayısı 1'e düşünce, bu çağrılar ve iş yükü drastik şekilde azalır.

4. Verimlilik (Efficiency):

- Kullanıcı her yeni sekmeyi açtığında tekrar login ekranıyla boğuşmaz. İş akışı kesilmez.